

LAUNCH REQUIREMENTS

What's Needed Before Public Launch

A consolidated, repo-grounded punch list — what's built, what's genuinely blocking, and what's a deliberate post-launch call. Distinct from the Staging & Production Deployment Plan (deployment mechanics); this is the "are we actually ready" checklist.

Companion document: [deployment-plan.pdf](#) (how to deploy) — this document covers what must be true before you tell real users to sign up and pay.

Source: [docs/requirements-validation.md](#), [docs/checklists.md](#), [SECURITY.md](#), [docs/backup-restore.md](#), live repo inspection (this pass)

1 · Where things actually stand

All eight planned build phases (auth/RBAC, catalogue, content workflow, student learning, test engine, payments/entitlements, doubts/notifications/support, production readiness) are implemented, plus this engagement's additions (institutions/multi-tenancy, dual payment integration, marketing CMS, Course Studio). The platform is functionally complete for a single-institution or multi-institution marketplace launch. What's below is the gap between "feature-complete" and "safe to launch."

A note on the existing `docs/requirements-validation.md` : it's accurate for most rows but is now stale on at least one point — it lists "migrations  " under non-functional requirements, but a live check this pass found **no** `apps/api/prisma/migrations/` **directory exists at all** (see Section 2.1). Treat any document written before this pass as a snapshot, not current truth — this document supersedes it on infra-readiness specifically.

2 · Blocking — cannot launch without these

2.1 — Real Prisma migration history Engineering

Every schema change to date (all 8 phases plus this engagement) was applied via `prisma db push` against dev databases, never `prisma migrate dev`. There is no `apps/api/prisma/migrations/` folder. The deploy tooling's `migrate.sh` runs `prisma migrate deploy`, which replays that history — with zero migrations, a fresh Staging/Production database never receives the schema. **Must run `prisma migrate dev --name init` once against a disposable database and commit the result before any real deploy.**

2.2 — First Super Admin bootstrap Engineering

`seed.ts` correctly seeds zero demo accounts when `NODE_ENV=production` — but that also means a fresh Staging/Production database has **no staff accounts at all**, and every admin action requires already being logged in as staff. No bootstrap script exists yet. Needs a one-time script (or manual DB insert) to create exactly one real Super Admin before the platform is usable at all.

2.3 — Legal pages: Terms of Service, Privacy Policy, Refund/Cancellation Policy You / Legal

New finding this pass: no Terms, Privacy Policy, or Refund/Cancellation page exists anywhere in `apps/web` — confirmed via a direct search of the app directory. This is not optional:

- **Razorpay requires** a live merchant account to link to publicly-visible Terms, Privacy, and Refund/Cancellation policy pages before approving live-mode KYC — without these, you cannot legally accept real payments in production, only test-mode ones.
- India's IT Rules / Consumer Protection (E-Commerce) Rules expect a clear refund/cancellation policy and grievance contact for any platform taking consumer payments.

This needs real legal content from you (or counsel) — I can build the pages once you provide (or approve drafted) copy, but the policy content itself is a business/legal decision, not an engineering one.

2.4 — A real Razorpay account, KYC-approved for live mode You / Business

Everything today runs against test-mode keys (or no keys, dev-fallback). Live payments need a Razorpay business account, completed KYC (bank account, PAN, business registration proof — takes Razorpay's team real review time, budget days not hours), and the legal pages above linked from your site before they'll approve it.

3 · Security — tracked in SECURITY.md, still open

Per the repository's own security checklist, these are known and tracked, not new findings — but worth surfacing here since they are launch-blocking for a payments-and-PII platform, not nice-to-haves:

- ☐ Malware scanning on upload-complete (currently a "basic" MIME/size check only; `ASSET_SCAN_PROVIDER` extension point exists for ClamAV/an external scanner)
- ☐ External penetration test — IDOR / role-escalation / upload abuse / webhook replay — before public launch, per `SECURITY.md`'s own stated requirement
- ☐ Nonce-based CSP (drop `'unsafe-inline'` for scripts) — defense-in-depth, not currently exploitable given React's escaping, but tracked as a follow-up
- ☐ Container image scanning in CI (dependency scanning via `pnpm audit` already runs; image scanning does not yet)

Already solid (verified present in code, not just documented): argon2id password hashing, refresh-token rotation with reuse detection, server-side RBAC + assignment-scope enforcement on every protected endpoint, OTP/login rate-limiting + lockout, TOTP MFA + AAL2 step-up for high-risk actions, HMAC-verified + idempotent payment webhooks, signed short-lived upload/playback URLs + watermark, append-only audit log, CSP/HSTS/X-Frame-Options headers, health+metrics+correlation-id logging.

4 · QA & accessibility — tracked, still open

- ☐ Formal accessibility audit (current state: Tailwind mobile-first + AA color tokens + visible focus states applied throughout, but never independently audited)
- ☐ Performance / Core Web Vitals review — no measurement has been taken
- ☐ Broaden automated test coverage beyond the current critical-path set (policy engine 16/16, scoring 9/9, one e2e auth spec) — acceptable for now, thin for a payments platform at scale
- ☐ Restore-drill: per `docs/backup-restore.md`, a full backup-restore-into-isolated-db drill is required before launch, then quarterly — has not yet been run (no production database exists yet to drill against)

5 · Business & content readiness You

All of these are yours to close — engineering can't do them from inside the codebase:

- ☐ At least one real, fully-published course (with real curriculum, real pricing, real published lessons) — the platform currently has one small real demo course (BPSC Prelims) plus test artifacts; nothing production-grade yet
- ☐ Real domain(s) purchased/DNS-configured for the four subdomains per tier (see deployment-plan.pdf §2)
- ☐ Real SMS provider (MSG91) account + DLT-approved template — OTP is currently console-log-only in dev; without this, real students can't receive OTPs
- ☐ Real transactional email provider account (current dev default is a local mailhog sink)
- ☐ Error tracking (Sentry) account — `SENTRY_DSN` env var is reserved in code but nothing is wired to a real project yet
- ☐ Support/grievance contact channel published (a real inbox for security@ and a general support contact — currently only a placeholder in SECURITY.md)
- ☐ Brand/domain/trademark clearance — confirm no conflicting "RajyaRank" registration exists in your target markets

6 · One-page launch punch list

#	Item	Owner	Status
1	Generate real Prisma migration history	Engineering	☐ Open
2	Build Super Admin bootstrap script	Engineering	☐ Open
3	Terms / Privacy / Refund pages (content + build)	You + Engineering	☐ Open
4	Razorpay live KYC approval	You	☐ Open
5	Malware scan on uploads (real provider)	Engineering	☐ Open
6	External penetration test	You (procure) + Engineering (fix findings)	☐ Open
7	Accessibility + performance audit	You (procure) or Engineering	☐ Open
8	Backup/restore drill	Engineering	☐ Open
9	Real course content populated	You (Academic content)	☐ Open
10	Domain/DNS + SMS/email/Sentry accounts	You	☐ Open
11	Staging deploy verified end-to-end (see deployment-plan.pdf)	Engineering	☐ Open

Items 1, 2, 5, 8, 11 are things I can build/run directly. Items 3 (content), 4, 6, 7 (procurement), 9, 10 need your input or a decision only you can make — happy to start on any of these the moment you're ready.